



河北建筑工程学院

HeBei University Of Architecture

学士学位毕业设计

华锐 SL1500 型双馈式风机齿轮箱 智能健康管理体系

姓 名 刘淑杨

学 号 20223090239

院 系 信息工程学院

专 业 计算机科学与技术

班 级 计 224

指导教师 董颢霞

毕业设计原创性声明

本人所提交的毕业设计《**华锐 SL1500 型双馈式风机齿轮箱智能健康管理体系**》，是在导师的指导下，独立进行研究工作所取得的原创性成果。除文中已经注明引用的内容外，本毕业设计不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中标明。

本声明的法律后果由本人承担。

毕业设计作者（签名）：

年 月 日

指导教师确认（签名）：

年 月 日

毕业设计版权使用授权书

本毕业设计作者完全了解河北建筑工程学院有权保留并向国家有关部门或机构送交毕业设计的复印件和磁盘，允许毕业设计被查阅和借阅。本人授权河北建筑工程学院可以将毕业设计的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或其它复制手段保存、汇编毕业设计。

保密的毕业设计在_____年解密后适用本授权书。

毕业设计作者（签名）：

年 月 日

指导教师（签名）：

年 月 日

摘 要

风电场运行规模扩大以后,运维人员最先感受到的不是单台机组数据变多,而是多台机组状态同时变化时难以及时判断重点风险。齿轮箱位于双馈式风机传动链中间,长期承受风速波动、启停冲击、温度变化和润滑状态变化的影响,容易出现齿面磨损、轴承损伤、油温偏高、油液污染和冷却异常等问题。如果故障只停留在报警列表中,运维人员往往还需要再翻看油温、振动、功率和历史检修记录,判断过程比较分散。基于这一问题,本设计围绕华锐 SL1500 型双馈式风机齿轮箱,完成状态监测、故障诊断、健康评分和运维建议的一体化展示。

系统实现时采用本地可运行的 Web 架构。后端使用 Flask 组织认证、风场、数据、报告、问答和运维接口,数据库使用 SQLite 保存用户、故障记录、参数配置和闭环处理信息;前端由 HTML、CSS 和 JavaScript 组成,利用 ECharts 展示趋势和统计图,用 Three.js 展示齿轮箱结构示意图,并通过 SSE 接收实时状态。项目代码中既有 DataAcquisitionSystem 生成的油温、振动、油液和功率测点,也有 WindDataRepository 对风机数据文件的整理结果,因此页面能够同时显示模拟实时数据和历史故障统计信息。

诊断模块没有只依赖单个阈值,而是把振动 RMS、峰值、峭度、峰值因子、脉冲因子、包络峰值、油温和油液 NAS 等指标放在一起判断。例如油温升高时,系统会继续查看振动和油液状态,避免把负荷变化造成的温升直接判断为严重故障;当振动和油液指标同时异常时,诊断结果会提高风险等级。最终页面会给出故障类型、严重程度、置信度、健康评分、RUL、诊断依据和维护建议,便于运维人员继续排查。

测试过程中,系统能够完成风场总览、单机详情、实时监测、故障诊断、故障代码库、故障数据管理、健康报告、参数设置、用户管理和智能运维诊断助手等功能。在本地演示环境下,前端页面可以随数据变化刷新油温、振动、功率、健康评分和 RUL,诊断结果也能写入故障记录并用于后续报告生成。目前系统仍属于毕业设计原型,真实应用时还需要接入现场 SCADA、CMS 和油液检测数据,并用实际故障样本继续校验模型参数。

关键词: 华锐 SL1500; 双馈式风机; 齿轮箱; 故障诊断; 健康管理

ABSTRACT

With the continuous expansion of wind power capacity, wind farms require higher reliability, availability and intelligent operation capability. As a key component in the transmission chain of a doubly-fed wind turbine, the gearbox is exposed to alternating loads, impact loads, temperature fluctuations and complex environmental conditions. It is prone to gear wear, pitting, bearing damage, abnormal oil temperature, lubricant deterioration and cooling system faults. If gearbox faults are not detected in time, they may lead to power limitation, unplanned shutdown and expensive component replacement.

Taking the gearbox of the Sinovel SL1500 doubly-fed wind turbine as the research object, this thesis designs and implements an intelligent health management system. The system adopts Flask as the backend framework, SQLite as the database, and HTML/CSS/JavaScript as the frontend technology stack. ECharts, Three.js and Server-Sent Events are used for real-time monitoring, fault trend analysis and digital twin visualization. The backend integrates data acquisition, wind turbine data repository, vibration feature extraction, fault diagnosis, health score calculation, remaining useful life estimation, fault record management, report generation and maintenance question answering.

The system extracts vibration features such as RMS, peak value, kurtosis, crest factor, impulse factor and envelope peak. It combines gearbox oil temperature, vibration intensity, oil quality and historical fault information to calculate risk factors. Based on a rule-based diagnostic library, the system identifies typical gearbox faults and outputs health score, RUL, diagnostic basis and maintenance suggestions. A local expert knowledge base and an optional large language model interface are also designed for intelligent operation and maintenance assistance.

The test results show that the system can realize wind farm overview, turbine detail monitoring, real-time data display, fault diagnosis, fault code management, fault data management, health report generation, parameter setting, user management and intelligent maintenance Q&A. It provides intuitive status perception and decision support for wind farm operators. The current system is still a graduation design prototype, and future work will focus on real SCADA/CMS/oil monitoring data access and machine learning model validation based on real fault samples.

Key words: SL1500; doubly-fed wind turbine; gearbox; fault diagnosis; health management

目 录

第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 传统故障诊断方法	1
1.2.2 智能算法故障诊断方法	1
1.2.3 现有技术的局限性	2
1.3 本文研究内容	2
1.4 技术路线	2
第 2 章 系统需求分析	3
2.1 业务需求分析	3
2.2 功能需求分析	3
2.2.1 健康监测需求	3
2.2.2 智能诊断需求	4
2.2.3 数据管理与报告需求	4
2.3 非功能需求分析	4
2.4 数据需求分析	4
第 3 章 相关技术与理论基础	5
3.1 齿轮箱典型故障机理	5
3.2 多源状态监测技术	5
3.3 振动特征提取方法	5
3.3.1 时域特征	5
3.3.2 包络特征	6
3.4 风险评分与健康评估方法	6
3.5 Web 系统开发技术	6
第 4 章 系统总体设计	8
4.1 系统架构设计	8
4.2 数据库设计	9
4.2.1 数据表设计	9
4.2.2 ER 图设计	10
4.3 后端接口设计	11
4.3.1 接口设计原则	11
4.4 诊断流程设计	12

4.5 前端页面设计.....	13
4.5.1 前端页面结构设计.....	13
4.5.2 人机交互界面设计.....	15
第5章 系统详细实现.....	17
5.1 数据采集与融合模块实现.....	17
5.2 故障诊断算法模块实现.....	18
5.2.1 振动特征提取实现.....	18
5.2.2 风险评分实现.....	20
5.3 风场总览与单机详情实现.....	20
5.3.1 风场总览页面.....	20
5.3.2 单机详情页面.....	22
5.3.3 HMI 人机交互页面.....	22
5.4 故障代码库与数据管理实现.....	25
5.5 AI 运维问答模块实现.....	26
5.6 健康报告与工单闭环实现.....	28
第6章 系统测试与结果分析.....	31
6.1 测试环境.....	31
6.2 功能测试.....	31
6.3 诊断场景测试分析.....	32
6.4 结果评价.....	32
第7章 总结与展望.....	33
7.1 研究总结.....	33
7.2 存在不足.....	33
7.3 后续展望.....	33
参考文献.....	34
致谢.....	35

第 1 章 绪论

1.1 研究背景与意义

风电场运维的难点在于机组数量多、运行环境变化快、故障处理窗口短。山地、戈壁、海岸和高寒地区的风机长期受到风速波动、温度变化、沙尘、湿度和并网冲击影响，关键部件一旦停机维修，往往会牵涉吊装、备件和发电损失。因此，毕业设计不能只做一个数据展示页面，还需要把状态监测、故障判断和处置建议串成完整流程。

华锐 SL1500 型双馈式风机的齿轮箱承担低速大扭矩到高速传动的转换任务。齿轮、轴承、润滑油路、冷却系统和箱体结构共同工作，任何一个环节异常都可能反映到油温、振动或油液颗粒度上。结合项目页面和后端诊断逻辑，本设计重点关注油温偏高、轴承振动升高、齿轮啮合异常、油液污染、冷却能力不足和基础松动等场景。

单纯依靠人工巡检和固定阈值报警时，早期故障容易被工况变化掩盖。以油温为例，温度升高可能来自环境温度、负荷提升、冷却风扇异常、润滑油劣化或轴承摩擦增加。如果系统只提示“油温高”，运维人员仍然需要逐项排查。因此，本设计在页面中同时展示油温、振动、油液 NAS、功率、健康评分和历史故障记录，让诊断结论有可追溯的数据依据。

本课题的目标是把分散的齿轮箱状态信息整理到同一个平台中。风场总览用于快速识别重点机组，单机详情用于查看当前运行状态，故障诊断模块用于生成风险等级和维护建议，健康报告模块用于形成阶段性汇总，智能运维诊断助手则用于回答排查步骤类问题。这些功能共同服务于一个目标：让运维人员能够从“发现异常”继续走到“判断原因”和“安排处理”。

1.2 国内外研究现状

1.2.1 传统故障诊断方法

从已有工程经验看，齿轮箱健康管理通常会同时关注振动、温度、油液和历史报警。振动信号适合反映齿轮啮合、轴承冲击和基础松动；温度数据适合观察润滑、冷却和负荷变化；油液数据能够反映磨粒和污染情况；历史报警则提供故障发生频次和持续时间。本系统没有把这些信息拆成孤立页面，而是在风场总览、单机详情和诊断结果中进行关联展示。

1.2.2 智能算法故障诊断方法

SCADA 数据在实际风场中最容易获取，字段通常包括风速、功率、转速、油温、变桨角、机舱位置和报警状态。这类数据采样频率不高，不能完全描述高频机械冲击，但适合做运行趋势判断。因此项目中将 SCADA 类字段作为页面基础状态，再用振动和油液指标补充机械故障信息。

1.2.3 现有技术的局限性

油液状态对齿轮箱诊断很有帮助。NAS 等级升高通常说明油液污染或磨粒增多，和油温、振动结合后，可以更好地区分润滑问题、轴承磨损和齿面损伤。考虑到油液数据并非实时高频采集，系统把它作为风险因子参与评分，而不是单独作为故障判定依据。

1.3 本文研究内容

本设计基于现有项目代码和风机数据文件完成，写作内容主要围绕已经实现的页面、接口、数据表和诊断逻辑展开。主要工作如下：

第一，梳理齿轮箱常见故障和运维流程。结合齿轮、轴承、润滑、冷却、轴系和基础结构相关故障，明确系统需要支持风场总览、单机详情、实时监测、故障诊断、健康报告和运维问答等功能。

第二，完成多源数据接入。后端通过 DataAcquisitionSystem 生成 TEMP_GBX_OIL、VIB_HS_BRG、OIL_NAS 和 P_ACTIVE 等测点数据，同时通过 WindDataRepository 读取风机数据摘要，把自由报表、实时数据、故障统计和故障信息统计整理成前端可直接使用的机组状态。

第三，完成齿轮箱诊断流程。系统先计算振动和油液相关特征，再结合油温、振动 RMS、油液 NAS 和历史故障信息生成风险因子，最后输出故障类型、严重程度、置信度、健康评分、RUL、诊断依据和维护建议。

第四，完成 Web 健康管理平台。系统使用 Flask 提供认证、数据、风场、报告、问答和运维接口，前端使用 HTML、CSS、JavaScript、ECharts 和 Three.js 实现登录、风场总览、单机详情、HMI 人机交互、故障数据、健康报告和智能运维诊断助手等页面。

第五，进行系统功能测试。测试时重点检查机组切换、实时数据刷新、诊断场景切换、故障记录保存、CSV 导出、报告生成和问答流式返回，观察前端页面和后端接口是否能够保持一致。

1.4 技术路线

本设计的技术路线从项目实现出发，可概括为需求分析、数据接入、特征计算、诊断评分、接口设计、前端展示和测试验证。系统先整理油温、振动、油液、功率、风速、转速和故障码等数据，再进行字段统一和简单预处理；随后提取振动特征并计算风险因子；最后把诊断结果、健康评分、RUL、复检周期和工单建议展示到页面中。

这条路线的重点是把机械故障含义落到软件功能上。例如振动 RMS 不只是一个数值，还会影响风险等级、健康评分和诊断建议；故障记录也不只是列表数据，还会继续用于健康报告和闭环处理。因此论文内容既说明齿轮箱指标的含义，也说明这些指标在代码和页面中如何被使用。

第 2 章 系统需求分析

2.1 业务需求分析

从使用场景看，运维人员需要同时关注多台机组。如果油温、振动、故障码和检修记录分散在不同文件或页面中，现场判断会比较慢。因此系统首页先给出风场级总览，再允许用户进入单机详情、故障诊断和报告页面，尽量减少来回查找数据的过程。

按照日常运维流程，系统需要支持以下操作：查看风场所有机组运行状态；识别告警、故障、限功率、待风和维护停机状态；查看单机油温、振动、功率、风速、健康评分和 RUL；根据故障码和阈值判断风险等级；生成维护建议；保存和导出故障记录；生成健康评估报告；通过问答页面获取排查步骤；模拟工单处理闭环。

2.2 功能需求分析

表 2.1 系统功能需求汇总

功能模块	主要功能	项目代码对应
用户与权限	登录、注册、角色显示、管理员默认账号初始化	auth_route.py、database.py
健康监测	风场总览、单机详情、实时状态、振动波形、SSE 推送	windfarm_route.py、data_route.py
智能诊断	特征提取、风险评分、故障分类、健康评分和 RUL 估计	ml_models.py
数据管理	故障记录列表、历史故障统计、CSV 导出	data_route.py、wind_data_repository.py
运维问答	本地知识库检索、实时工况分析、可选大模型增强	rag_service.py、qa_route.py
报告与工单	健康报告、复检建议、P1/P2 工单模拟	report_route.py、ops_route.py

2.2.1 健康监测需求

系统功能需求包括用户管理、健康监测、智能诊断、数据管理、报告生成、参数配置和运维问答七个方面。

用户管理模块包括用户登录、注册和角色显示。后端通过 auth_route.py 提供登录注

册接口，数据库中的 User 表保存用户名、密码哈希和角色。

健康监测模块包括风场总览、单机详情和实时监测。风场总览展示时间可利用率、能量可利用率、平均油温、平均风速、总功率、平均健康评分、告警数量和运行机组数。单机详情展示有功功率、风速、发电机转速、变桨角、温度、寿命、部件状态、集群对标和告警记录。实时监测页面通过 SSE 接收状态和振动波形。

2.2.2 智能诊断需求

智能诊断模块包括故障诊断、故障代码库和 AI 运维问答。故障诊断支持正常、齿轮磨损、点蚀剥落、断齿、轴承磨损、轴承点蚀、高速轴承过温、齿轮箱油温过高、油液污染、轴系不对中、齿轮啮合异常、基础松动和冷却故障等场景。故障代码库包括 GBX-001 至 GBX-015 等齿轮箱关联故障，并能结合历史故障统计动态补充现场故障码。

2.2.3 数据管理与报告需求

数据管理模块展示诊断记录和历史故障，支持 CSV 导出。报告模块根据当前状态和历史趋势生成健康评分、关键指标、诊断结论、维护计划和工单建议。参数配置模块展示监测采集、齿轮箱阈值、寿命评估、诊断模型和报警策略参数。运维问答模块基于本地知识库和实时状态生成建议。

2.3 非功能需求分析

系统设计时主要考虑三个方面。第一是可用性，关键指标要在页面上直接可见，不能让用户在多个页面中反复查找。第二是可扩展性，后续如果接入真实 SCADA、CMS、油液监测或工单系统，后端接口和数据表需要有扩展空间。第三是可解释性，诊断结果不能只给出故障名称，还要说明风险因子、诊断依据、健康评分、RUL 和建议措施。

本系统目前以本地演示为主，因此数据库选用 SQLite。这种方案部署简单，便于答辩演示和数据迁移。如果后续进入风场实际应用，可以将数据库迁移到 MySQL、PostgreSQL 或时序数据库，并增加权限审计、日志留存和消息队列等功能。

2.4 数据需求分析

项目数据主要来自两部分。一部分由数据采集模块模拟生成，包括齿轮箱油温、高速轴承振动、油液颗粒度和有功功率；另一部分来自风机数据文件夹中的统计结果，包括自由报表、实时数据、故障统计和故障信息统计。系统将这些字段整理成统一机组快照，供风场总览、单机详情、诊断和报告模块调用。

第3章 相关技术与理论基础

3.1 齿轮箱典型故障机理

风机齿轮箱故障可按照部件和机理分为齿轮故障、轴承故障、润滑故障、冷却故障、轴系故障和结构故障。齿轮故障包括齿面磨损、点蚀剥落、断齿和啮合异常。齿面磨损通常由润滑不足、载荷波动、异物颗粒和长期疲劳引起，会导致啮合频率附近能量升高；点蚀剥落属于接触疲劳损伤，常伴随冲击成分增强；断齿属于严重故障，可能出现周期性冲击并引起快速停机风险。

轴承故障包括磨损、点蚀、剥落和高速轴承过温。轴承磨损初期可能表现为振动 RMS 缓慢升高，点蚀和剥落则会导致峭度、峰值因子和包络峰值升高。高速轴承过温可能与润滑不足、游隙异常、冷却效率下降或测点异常有关。

润滑和冷却故障会对齿轮箱整体健康产生连锁影响。润滑油污染或劣化会降低油膜承载能力，加速齿轮和轴承磨损；冷却系统故障会使油温升高，导致油液黏度变化和热变形增加。轴系不对中和箱体基础松动则通常表现为低频振动增强、宽频振动升高和工况变化下振动波动。

3.2 多源状态监测技术

齿轮箱状态判断不能只看单一数据源。SCADA 数据适合观察油温、功率、风速、转速、变桨角和机舱位置等低频工况；CMS 振动数据更适合观察齿轮啮合、轴承冲击和结构松动；油液检测能够反映磨粒和污染变化；故障台账则记录历史报警和停机事件。本系统把这些信息放到同一条诊断链路中，减少人工比对。

在代码实现中，采集测点被统一命名为 TEMP_GBX_OIL、VIB_HS_BRG、OIL_NAS 和 P_ACTIVE，分别代表齿轮箱油温、高速轴承振动、油液颗粒度和有功功率。这种命名方式使前端页面、诊断模块和报告模块能够使用同一套字段，避免不同数据来源之间反复转换。

3.3 振动特征提取方法

3.3.1 时域特征

振动数据在本系统中主要用于识别冲击和能量变化。RMS 反映整体振动强度，峰值反映瞬时冲击，峭度用于观察脉冲特征，峰值因子和脉冲因子用于衡量异常冲击的突出程度，包络峰值用于辅助判断调制冲击。这些特征会参与风险评分，而不是单独显示在后台计算中。

为了完成本地演示，振动波形由基频、轴频、啮合频率和随机噪声叠加生成，部分故障场景会加入

冲击成分。前端实时监测页面接收这些数据后刷新波形和指标卡片，用户可以直观看到不同故障场景下振动特征的变化。

3.3.2 包络特征

包络分析用于突出齿轮和轴承局部损伤产生的调制冲击。当齿面点蚀、轴承剥落或断齿出现时，包络峰值与冲击类指标会同步升高，可作为系统规则诊断的重要依据。

3.4 风险评分与健康评估方法

表 3.1 齿轮箱健康评估主要风险因子

风险因子	含义	诊断作用
振动 RMS	反映整体振动能量	识别磨损、松动和运行异常
峭度	反映冲击尖峰程度	识别轴承剥落、齿面点蚀等早期冲击
峰值因子	峰值与有效值比值	增强对局部冲击的敏感性
包络峰值	反映调制冲击强度	辅助判断轴承和齿轮局部损伤
齿轮箱油温	反映热状态和冷却效率	识别过温和冷却系统异常
油液 NAS	反映润滑油污染程度	识别润滑劣化和磨粒风险

系统将多个特征和工况指标归一化为风险因子，包括振动 RMS、冲击峭度、峰值因子、包络峰值、齿轮箱油温和油液 NAS。每个风险因子根据经验阈值映射到 0 到 1 的区间，再采用加权求和得到综合风险值。振动 RMS 和冲击峭度权重较高，油温和油液状态作为重要补充。

健康评分由综合风险值和故障严重程度共同决定。风险越高，健康评分越低；若故障等级为警告或严重，则进一步扣减评分。RUL 根据健康评分、风险值和故障严重程度估计，健康评分高、风险值低时 RUL 较长；严重故障会显著缩短 RUL。虽然该方法属于规则型估计，但能够在毕业设计阶段形成可解释的健康管理流程。

3.5 Web 系统开发技术

系统采用 Flask 框架构建后端服务。Flask 具有轻量、灵活、易扩展的特点，适合毕业设计原型开发。数据库使用 Flask-SQLAlchemy 管理，表结构包括用户表、故障记录表、机组资产表和系统配置表。前端使用 HTML、CSS 和 JavaScript 实现，ECharts 用于趋势图、饼图和柱状图展示，Three.js 用于齿轮箱三维结构示意。系统通过 Server-Sent Events

实现实时状态推送，使页面能够持续刷新油温、振动和波形数据。

从系统实现角度看，后端、数据库和前端并不是孤立模块，而是共同支撑齿轮箱健康管理流程。后端负责把原始状态数据转化为结构化诊断结果，数据库负责保存用户、故障记录、配置参数和报告历史，前端则将复杂指标转化为运维人员能够快速理解的页面信息。三者结合后，系统才能同时具备实时监测、历史追溯和辅助决策能力。

在毕业设计实现中，技术选型更强调可实现性和可演示性。**Flask** 便于快速组织接口，**SQLite** 适合轻量化部署，**ECharts** 能够直观展示趋势和分布，**SSE** 可以降低实时刷新实现复杂度。该组合虽然不等同于生产级风电场平台，但能够覆盖原型系统的核心功能，并为后续接入真实 **SCADA**、**CMS** 和油液监测数据预留扩展空间。

Web 系统开发技术的作用还体现在系统集成能力上。齿轮箱健康管理涉及实时数据、故障规则、历史记录、报告输出和用户交互等多个环节，如果缺少统一的平台载体，诊断结果难以被运维人员持续使用。通过 **Web** 系统将各类功能组织在同一界面中，可以提高系统演示的完整性，也便于后续扩展移动端访问、权限控制和远程运维功能。

综上，本文后续系统设计将围绕数据流、业务流和界面流展开。数据流解决齿轮箱状态如何采集、处理和保存的问题，业务流解决故障如何诊断、记录和闭环的问题，界面流解决运维人员如何查看、理解和处理结果的问题。三类流程相互配合，构成系统总体设计和详细实现的基础。

第 4 章 系统总体设计

4.1 系统架构设计

系统功能架构如图 4.1 所示。系统按表现层、接口层、业务服务层和数据层组织，前端负责状态展示与交互操作，后端负责数据处理、诊断计算、报告生成和知识问答，数据层为诊断模型和页面展示提供基础支撑。

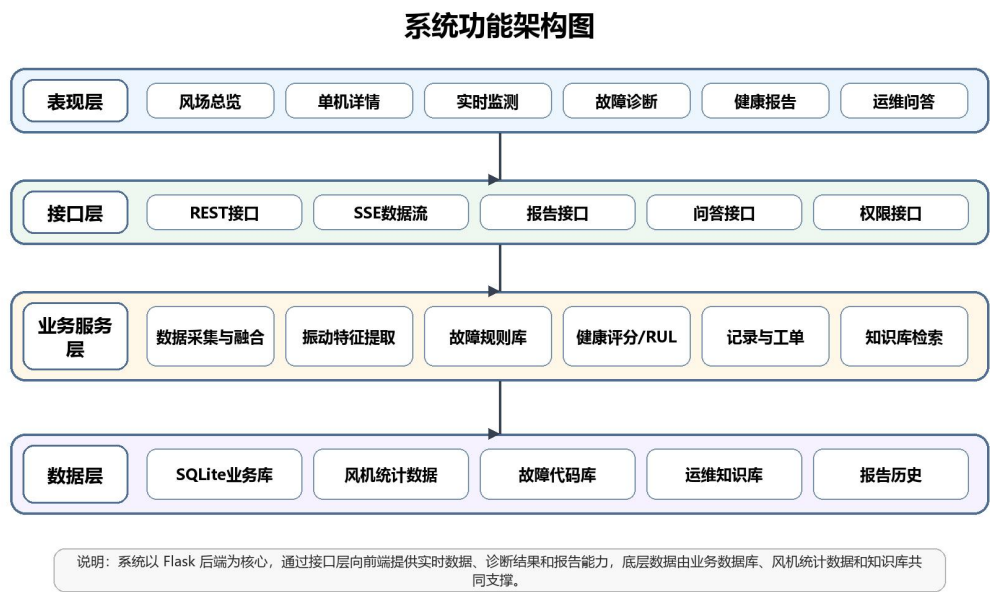


图 4.1 系统功能架构图

系统总体采用 B/S 架构，用户通过浏览器访问前端页面，前端通过 HTTP 接口和 SSE 通道与 Flask 后端通信。后端由应用入口、路由层、服务层、数据模型层和数据仓库层组成。应用入口负责创建 Flask 应用、配置数据库和注册蓝图；路由层提供认证、数据、风场、报告、问答、设置和运维接口；服务层负责数据模拟、数据融合、诊断计算和知识问答；数据模型层负责数据库表定义；数据仓库层负责读取风机数据摘要文件。

前端按业务区域划分为健康监测区、智能诊断区和系统配置区。健康监测区包括风场总览、单机详情和实时监测；智能诊断区包括故障诊断、运维辅助问答、故障代码库、故障数据和健康报告；系统配置区包括参数设置。另有独立 HMI 页面用于展示单机齿轮箱人机交互界面。

4.2 数据库设计

表 4.1 数据库表结构说明

数据表	关键字段	用途
User	username、password、role	保存用户账号、密码哈希和角色
FaultRecord	unit_id、timestamp、fault_type、severity、probability、status	保存诊断结果和处理状态
TurbineAsset	unit_id、number、model、design_life_years	保存风机资产基础信息
SystemConfig	config_key、config_value、description	保存油温、振动、油液等阈值配置

系统数据库采用 SQLite。User 表保存用户信息，包括用户名、密码和角色；FaultRecord 表保存诊断记录，包括机组编号、时间戳、故障类型、严重程度、置信度、建议措施和处理状态；TurbineAsset 表保存机组资产信息，包括机组编号、数字编号、型号、位置、状态、设计寿命和投运日期；SystemConfig 表保存系统参数，包括配置键、配置值和描述。

该数据库设计虽然较轻量，但能够覆盖毕业设计原型的核心数据需求。用户表支撑登录和权限展示；故障记录表支撑诊断结果落库、列表查看和 CSV 导出；机组资产表为后续真实风场资产管理预留空间；系统配置表用于保存油温、振动、油液等阈值参数。

4.2.1 数据表设计

数据库设计围绕用户、故障记录、参数配置和闭环处理展开。User 表用于保存用户账号、密码哈希和角色信息，支撑登录与权限控制；FaultRecord 表保存诊断产生的故障类型、严重程度、置信度、处理状态和建议措施；FaultClosure 表记录故障闭环过程，包括处理人、处理说明、复测结果和关闭时间；配置表则保存油温、振动、油液等阈值。

在业务流程上，诊断模块生成结果后会写入故障记录表，前端故障数据页面从该表读取记录并展示；当运维人员完成处理后，系统将处理信息写入闭环表，并更新故障记录状态。健康报告模块读取实时状态、历史故障和闭环记录，生成面向运维人员的综合报告。这样的数据库关系能够支持从报警发现到处理反馈的完整闭环。

表 4.2 数据库表关系说明

数据表	主要字段	业务作用
User	id、username、password_hash、role	保存用户和角色权限
FaultRecord	unit_id、fault_type、severity、status	保存诊断结果和处理状态
FaultClosure	record_id、handler、action、result	保存故障闭环处理信息
SystemConfig	threshold_name、value、updated_at	保存阈值和系统参数
ReportHistory	report_id、unit_id、summary、created_at	保存健康报告摘要

4.2.2 ER 图设计

系统数据库围绕用户、风机资产、故障记录、诊断结果、健康报告和系统参数展开。风机资产表是业务核心，故障记录、诊断结果和健康报告均通过机组编号与其关联，系统参数表为阈值配置和诊断规则提供支撑。

ER 图设计强调数据之间的业务关系。用户表对应系统访问主体，风机资产表对应被监测对象，故障记录表对应诊断事件，诊断结果和健康报告对应分析输出，系统参数表对应规则配置。通过这些实体关系，系统能够把一次故障诊断从页面操作转化为可保存、可查询、可统计的业务数据。

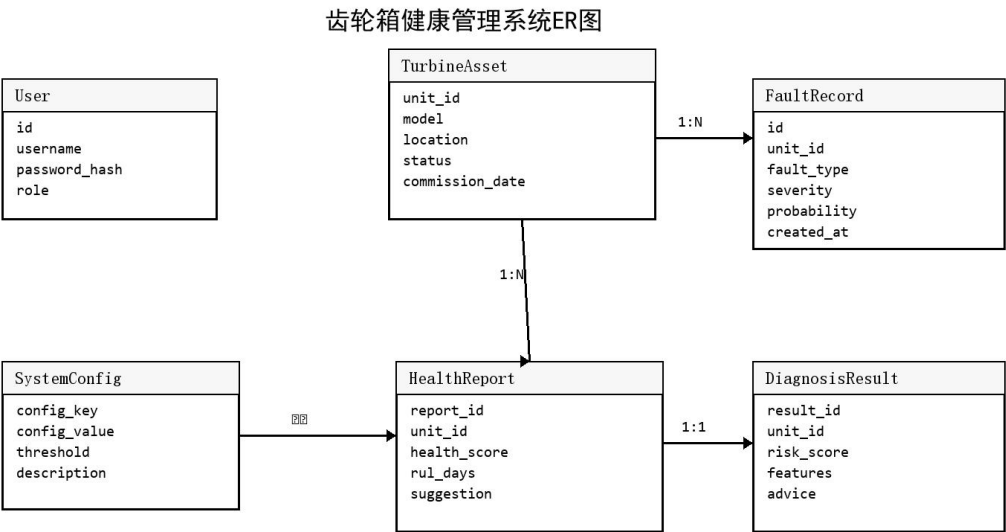


图 4.2 系统数据库 ER 图

4.3 后端接口设计

表 4.3 系统主要后端接口

接口类别	典型接口	功能说明
数据接口	/api/data/status、/api/data/diagnosis、 /api/data/trend	状态采集、诊断计算和趋势分析
风场接口	/api/windfarm/overview、 /api/windfarm/turbine/<unit_id>	风场总览和单机详情
报告接口	/api/report/generate、 /api/report/health-summary	生成健康报告和摘要
问答接口	/api/qa/ask	运维知识问答和实时状态分析
运维接口	/api/ops/workorders、 /api/ops/alarm-policy	工单模拟、报警策略和模型验证

系统后端接口按蓝图划分。数据接口 /api/data 提供振动波形、实时状态、故障代码、诊断结果、故障记录、故障记录导出、趋势分析和 SSE 数据流。风场接口 /api/windfarm 提供风场总览、单机详情、参数分组和风场流式数据。报告接口 /api/report 提供健康报告和报告历史。问答接口 /api/qa 提供运维问题回答。运维接口 /api/ops 提供 SCADA 状态、工单、模型验证、审计日志、趋势对比和报警策略。认证和设置接口分别负责登录注册和系统配置。

这种接口组织方式使系统业务边界较清楚。实时监测和诊断相关功能集中在数据接口中，风场和单机展示集中在风场接口中，报告和问答作为独立能力存在，便于后续扩展。

4.3.1 接口设计原则

后端接口按业务域划分为认证接口、数据接口、风场接口、报告接口、问答接口、设置接口和运维接口。认证接口负责登录、注册和当前用户信息；数据接口负责实时状态、振动数据、故障诊断、故障记录和故障代码库；风场接口负责风场总览和单机详情；报告接口负责生成健康报告；问答接口负责运维知识问答；设置接口负责阈值和用户管理；运维接口负责工单和报警策略。

接口返回数据尽量采用统一的 JSON 结构，前端页面只关注字段含义，不关心后端内部计算过程。这样一方面可以降低前端复杂度，另一方面也便于后续把规则诊断模块替换为机器学习服务或独立微服务。接口设计中还保留了 SSE 数据流，用于实时推送状态变化，增强监测页面的实时感。

表 4.4 后端接口分组说明

接口分组	代表路径	功能说明
认证接口	/api/auth/login	用户登录和身份验证
数据接口	/api/data/status	实时状态、振动和诊断数据
风场接口	/api/windfarm/overview	风场总览和单机详情
报告接口	/api/report/generate	生成健康报告
问答接口	/api/qa/ask	运维知识问答
运维接口	/api/ops/workorders	工单模拟和闭环处理

4.4 诊断流程设计

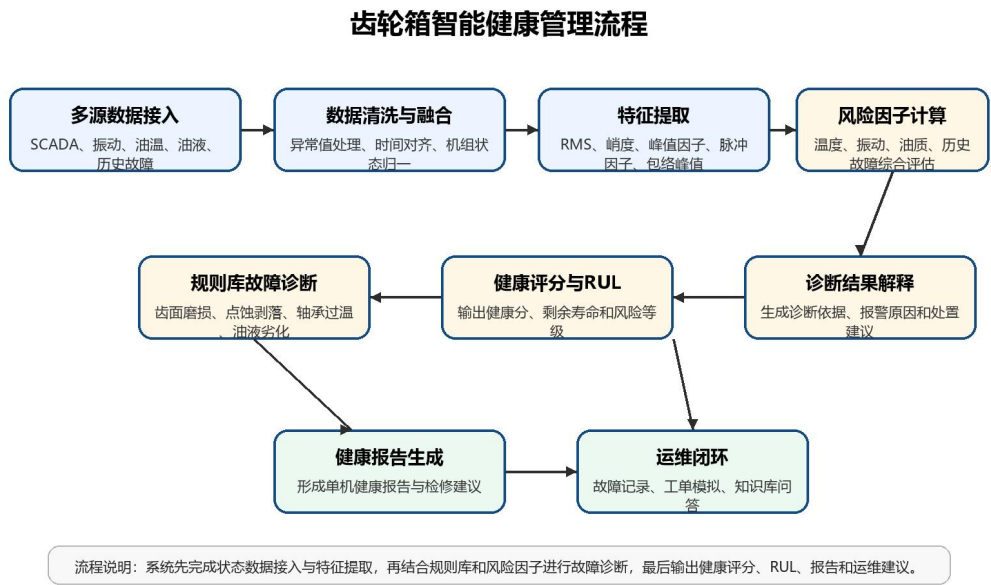


图 4.3 齿轮箱智能健康管理流程图

系统诊断流程如图 4.3 所示。该流程以多源状态数据为输入，经过数据清洗、特征提取、风险因子计算和规则库诊断后，形成健康评分、RUL 估计、诊断依据与运维建议，并通过健康报告、故障记录和工单模拟实现闭环管理。

系统诊断流程从选择机组和诊断场景开始。后端获取当前机组状态，生成或读取振动波形，并根据场景对油温、振动 RMS、油液质量和原始波形进行调整。例如过温场景会提高油温，不对中场景会叠加低频振动，轴承点蚀场景会加入周期性冲击，断齿场景会加入更强冲击并提高振动和油液风险。

随后系统提取振动特征并计算风险因子。综合风险值输入故障分类规则，得到故障类型、关联部件、严重程度和诊断依据。系统进一步计算健康评分和 RUL，生成维护动作清单。若诊断结果为警告或严重，系统模拟生成检修工单信息，包括工单编号、优先级、责任班组、建议处置窗口和闭环流程。最后，诊断记录写入数据库，供故障数据管理页面查看和导出。

4.5 前端页面设计

4.5.1 前端页面结构设计

前端首页包含登录、注册和主应用界面。用户通过登录页进入系统，系统根据角色显示高级工程师或运维人员身份；注册页可选择运维人员或管理员角色。登录后默认进入健康总览，左侧导航按照健康监测区、智能诊断区和系统配置区分组，顶部显示当前模块说明、通知状态和用户身份。风场总览页面展示多台机组卡片，单机详情页面展示齿轮箱温度、寿命、部件状态、告警和集群对标。实时监测页面进一步改为多标签结构，包含运行概览、振动趋势、孪生与采集、故障统计等页面。

故障诊断页面提供场景选择和诊断按钮，运行后展示疑似故障类型、风险等级、关联部件、置信度、健康评分、RUL、特征值、风险因子、诊断依据、建议措施和工单信息。故障数据页面展示历史故障记录，报告页面展示健康评估结果，问答页面支持输入自然语言问题并获得排查建议。更新后的 HMI 页面增加二次权限登录，登录后进入独立的单机齿轮箱控制台，提供运行总览、状态寿命、透明模型、数据采集、测点信号、阈值设置、故障代码和临界警报等功能。

代码清单 4.1 主系统登录请求与角色状态保存

核心代码如下：

```
loginForm.addEventListener("submit", async (e) => {
  e.preventDefault();
  const response = await fetch("/api/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      username: usernameInput.value,
      password: passwordInput.value
    })
  });
  const data = await response.json();
  if (response.ok) {
    localStorage.setItem("currentUser", JSON.stringify(data.user));
    applyAuthority(data.user.role);
  }
});
```

```

document.getElementById("login-overlay").style.display = "none";
document.getElementById("main-app").style.display = "flex";
showPostLoginDefault();
    }
});

```

代码清单 4.1 位于 frontend/js/main.js。登录表单提交后，前端向/api/auth/login 发送账号和密码；认证成功后把用户信息写入 localStorage，并根据角色调用 applyAuthority 控制页面权限，最后隐藏登录遮罩并进入默认业务页面。该逻辑对应论文中“登录后进入健康总览、按照角色显示身份和权限”的交互设计。

代码清单 4.2 后端用户认证接口

核心代码如下：

```

@auth_bp.route("/login", methods=["POST"])
def login():
    data = request.json or {}
    username = (data.get("username") or "").strip()
    password = data.get("password") or ""
    user = User.query.filter_by(username=username).first()
    password_ok = False
    if user:
        try:
            password_ok = check_password_hash(user.password, password)
        except ValueError:
            password_ok = False
    if not password_ok and user.password == password:
        password_ok = True
        user.password = generate_password_hash(password)
        db.session.commit()
    if user and password_ok:
        return jsonify({"status": "success",
            "user": {"username": user.username, "role": user.role}})
    return jsonify({"status": "error", "message": "用户名或密码错误。"}), 401

```

代码清单 4.2 位于 backend/routes/auth_route.py。接口先查询用户，再校验密码哈希；若历史演示数据仍为明文密码，则在首次正确登录时自动迁移为哈希值。接口返回 username 和 role 供前端完成角色显示与权限控制，使登录功能既满足演示便利性，也保留基本的账号安全处理。

表 4.5 前端页面与交互功能对应

前端页面	交互入口	主要展示内容	前后端对应
风场总览	左侧导航-健康总览	风场统计、多机组卡片、健康矩阵	windfarm_route.py
单机详情	机组卡片下钻	油温、振动、RUL、部件状态和历史故障	windfarm_route.py

故障诊断	智能诊断-故障诊断	场景选择、诊断结果、依据和建议措施	data_route.py、ml_models.py
故障数据	智能诊断-故障数据	故障记录、筛选、处理状态和 CSV 导出	data_route.py、database.py
AI 问答	智能诊断-运维辅助问答	知识库检索、实时工况分析和排查建议	qa_route.py、rag_service.py
健康报告	智能诊断-健康报告	健康评分、关键指标、维护计划和报告历史	report_route.py
HMI 界面	单机详情-HMI 控制台，二次权限登录	运行总览、阈值设置、故障代码、测点和临界报警	hmi.html、settings_route.py、data_route.py

4.5.2 人机交互界面设计

人机交互界面设计遵循“总览发现异常、详情定位原因、诊断形成结论、报告完成闭环”的操作路径。页面左侧采用稳定导航分组，减少用户在不同功能间切换时的认知负担；顶部区域显示当前页面名称、登录用户和通知状态，使运维人员能够确认当前操作对象。关键指标采用卡片和颜色状态表达，异常指标使用醒目色提示，但不改变页面整体结构，便于在长时间监控场景下保持可读性。

HMI 页面更强调现场设备交互。该页面围绕单台齿轮箱展开，用户从单机详情页点击 HMI 运维控制台后，首先进入权限登录页面，确认账号、密码和角色级别，再进入控制台。登录后的 HMI 页面将齿轮箱油温、振动 RMS、健康评分、RUL、运行状态和故障风险集中到同一视图，并提供状态寿命、透明模型、数据采集、测点信号、阈值设置、故障代码和临界警报等菜单。相比普通管理页面，HMI 界面能够体现设备对象、运行状态和操作反馈之间的对应关系，是系统面向现场监控应用的重要补充。

在交互流程上，系统避免让用户在多个页面之间反复寻找同一机组信息。风场总览用于发现异常，单机详情用于查看设备指标，故障诊断用于形成结论，健康报告用于沉淀结果，HMI 页面用于表达现场监控对象。这样的页面组织方式可以让运维人员按照实际工作路径逐步深入，而不是在分散功能中被动查找数据。

在人机交互反馈方面，系统将报警状态、健康评分、RUL、故障类型和建议措施放在同一业务链路中呈现。用户既能看到实时指标，也能看到系统给出判断的依据和后续处理建议。对于齿轮箱这类高价值设备，仅显示单一报警数值并不足够，必须同时提供故障解释、处理优先级和闭环入口，才能体现智能健康管理系统的实际意义。

因此，本文前端设计并非单纯追求页面美观，而是围绕运维人员的判断效率展开。总览页面强调快速发现问题，详情页面强调指标解释，诊断页面强调结论可信度，HMI 页面强调现场设备对象感，报告页面强调结果沉淀。不同页面承担不同任务，能够减少信息堆叠造成的阅读负担。

第 4 章的总体设计为第 5 章实现提供了结构依据。后续实现章节将按照数据采集、故障诊断、风场与单机页面、数据管理、问答辅助和健康报告等模块展开，分别说明关键代码、系统截图和运行效果，使系统设计与实际开发结果能够相互对应。

第 5 章 系统详细实现

5.1 数据采集与融合模块实现

数据采集模块位于 `services/data_acquisition.py`。该模块定义了四类传感器：齿轮箱油温、高速轴承振动、油液颗粒度和有功功率。系统通过基础值、周期波动和随机噪声生成测点数据，并通过多次采样和均值滤波降低异常噪声影响。采集结果包括油温、振动 RMS、油液 NAS、功率、链路延迟、丢包率、采样率、采集状态、数据质量、在线传感器数量、健康评分、RUL 和告警列表。

代码清单 5.1 振动信号特征提取核心代码

核心代码如下：

```
def process_signals(signal_data):
    arr = np.array(signal_data, dtype=float)
    rms = float(np.sqrt(np.mean(arr ** 2)))
    peak = float(np.max(np.abs(arr)))
    mean = float(np.mean(arr))
    std = float(np.std(arr)) or 1e-6
    centered = arr - mean
    kurtosis = float(np.mean(centered ** 4) / (std ** 4))
    crest_factor = float(peak / rms) if rms > 0 else 1.0
    impulse_factor = float(peak / (np.mean(np.abs(arr)) or 1e-6))
    envelope = get_envelope(arr.tolist())
    envelope_peak = float(max(envelope)) if envelope else 0.0
    return {
        "rms": round(rms, 4),
        "kurtosis": round(kurtosis, 4),
        "peak": round(peak, 4),
        "crest_factor": round(crest_factor, 4),
        "impulse_factor": round(impulse_factor, 4),
        "envelope_peak": round(envelope_peak, 4),
    }
```

风机数据仓库模块位于 `services/wind_data_repository.py`。该模块读取 风机数据 `/wind_data_summary.json`，将自由报表、实时数据、故障统计和故障信息统计整合为统一机组快照。对于每台机组，系统计算功率、风速、发电机转速、齿轮箱油温、环境温度、振动 RMS、健康评分和 RUL。对于历史故障，系统根据故障码映射中文标签，并将故障码分为齿轮箱关联故障、运行事件和数据采集故障等类别。

系统运行后的风场健康总览页面如图 5.1 所示。该页面以矩阵形式展示多台机组的运行状态，并在顶部汇总时间可利用率、能量可利用率、平均油温、总有功功率、平均健康评分和告警机组数量。



图 5.1 风场健康总览页面

风场总览页面是运维人员进入系统后最先接触的页面，其设计目标是快速定位异常机组。页面中不同颜色代表不同运行状态，绿色表示正常运行，橙色和红色表示告警或故障，紫色表示维护停机。每个机组卡片展示功率、风速、油温、健康评分和运行状态，便于用户在一个页面中完成多机组横向比较。

系统还提供指标列表和健康矩阵两种浏览方式。健康矩阵适合快速扫描，指标列表适合查看详细数值。运维人员可以先通过矩阵发现异常机组，再进入单机详情页面查看趋势、故障代码和检修建议。该设计符合风电场运维中“先总览、再下钻、后处理”的工作习惯。

5.2 故障诊断算法模块实现

5.2.1 振动特征提取实现

故障诊断模块位于 `services/ml_models.py`。系统首先调用 `process_signals` 计算振动特征，包括 RMS、峭度、峰值、峰值因子、脉冲因子和包络峰值。包络估计通过信号和梯度构造近似解析信号，再进行滑动平均，用于捕捉冲击调制特征。

代码清单 5.2 故障诊断与健康评分调用逻辑

核心代码如下：


```

def run_diagnosis(unit_id="WTG-001", raw_signal=None, status=None):
    raw_signal = raw_signal or generate_demo_signal()
    features = process_signals(raw_signal)
    risk_score, risk_factors, snapshot = score_risk(features, status)
    fault = classify_fault(features, risk_score, risk_factors, status)
    health_score = calculate_health_score(risk_score, fault["severity"])
    rul = predict_rul(health_score, risk_score, fault["severity"])
    explanation = build_explanation(
        features, fault["severity"], rul, risk_score, risk_factors, fault
    )
    return {
        "unit_id": unit_id,
        "features": features,
        "fault_type": fault["type"],
        "health_score": health_score,
        "rul_days": rul,
        "explanation": explanation,
    }

```

随后系统调用 `score_risk` 计算风险因子。风险因子包括振动 RMS、冲击峭度、峰值因子、包络峰值、齿轮箱油温和油液 NAS。系统采用归一化函数将原始值映射到 0 到 1，再按权重计算综合风险值。分类函数 `classify_fault` 根据强制场景、油温风险、油液质量、冲击特征和振动特征识别故障类型。系统故障库覆盖正常运行、齿轮齿面磨损、齿面点蚀/剥落、齿轮断齿、轴承磨损、轴承点蚀/剥落、高速轴承过温、齿轮箱油温过高、润滑油污染/油液劣化、轴系不对中、齿轮啮合异常、箱体或基础松动和冷却系统故障。

健康评分由 `calculate_health_score` 生成，RUL 由 `predict_rul` 估计。系统还会调用 `build_explanation` 生成诊断说明，包括风险等级、风险分数、结论、诊断依据、主要风险因子、特征含义和 RUL 说明。最终 `run_diagnosis` 将所有结果封装为接口响应，并尝试写入 `FaultRecord` 数据表。

故障诊断页面如图 5.2 所示。用户可以选择机组和诊断场景，系统执行诊断后输出故障类型、风险等级、健康评分、RUL、诊断依据和建议措施。



图 5.2 故障诊断页面

故障诊断页面是系统的核心业务页面。用户可以选择不同机组，也可以选择正常、齿轮磨损、点蚀剥落、轴承过温、油液污染、冷却故障等场景进行演示。点击开始诊断后，前端调用后端诊断接口，后端完成振动特征提取、风险评分、故障分类和结果解释，然后将结构化结果返回页面展示。

诊断结果并不只显示故障名称，而是同时给出风险等级、部件位置、置信度、健康评分和剩余寿命。下方的诊断依据和建议措施用于解释为什么系统给出该结论，以及下一步应该如何排查。这样的展示方式能够增强诊断结果可信度，避免用户只看到一个缺少依据的告警标签。

5.2.2 风险评分实现

风险评分实现将振动、油温、油液等级和故障严重度映射为统一风险值，再计算健康评分和剩余寿命。该方法虽然属于规则融合模型，但输出过程清晰，适合毕业设计原型系统展示和运维解释。

5.3 风场总览与单机详情实现

5.3.1 风场总览页面

风场总览接口位于 `routes/windfarm_route.py`。当风机数据摘要文件存在时，系统优先使用真实文件数据；否则使用模拟数据。总览结果包括风场名称、系统名称、时间戳、可

利用率、平均温度、平均风速、总功率、平均健康评分、告警数量、运行机组数量、机组总数、状态统计和机组列表。

单机详情接口 `/api/windfarm/turbine/<unit_id>` 返回指定机组的运行指标、温度、寿命、部件状态、集群对标、控制模式、SCADA 时间和告警列表。寿命部分包括设计寿命、已运行年限、剩余设计年限、齿轮箱剩余寿命、健康评分、RUL、复检周期和复检建议。部件状态包括低速轴承、高速轴承、齿轮箱油温、振动 RMS 和润滑油。

故障数据管理页面如图 5.3 所示。该页面用于查看诊断记录、筛选故障状态、导出 CSV 并进入故障闭环处理。



图 5.3 故障数据管理页面

故障数据管理页面承担历史记录追踪功能。系统每次诊断产生的记录会写入数据库，页面按机组、严重程度、处理状态和关键词进行筛选。对于严重或待处理故障，运维人员可以进入详情查看诊断依据、建议措施和处理进度。

该页面还提供 CSV 导出功能，便于将故障记录用于论文测试、运维日报或后续数据分析。对于毕业设计而言，故障记录页面能够证明系统不是只做一次性诊断，而是具备数据积累和闭环管理能力。

单机齿轮箱详情页面如图 5.4 所示。页面从风场总览下钻到单台机组，集中展示齿轮箱油温、高速轴承温度、振动 RMS、油液 NAS 等级、健康评分、RUL、复检周期和历史故障记录。



5.3.2 单机详情页面

图 5.4 单机齿轮箱详情页面

单机详情页面体现了系统从风场级监控到设备级诊断的下钻能力。运维人员在风场总览中发现异常机组后，可以进入该页面查看机组的关键指标和部件状态。该页面还提供 HMI 控制台、故障诊断、实时趋势和健康报告入口，使单台机组相关功能形成集中工作台。

从健康管理角度看，单机详情页面能够把不同来源的数据组织到同一个设备视图中。油温和轴承温度用于判断热状态，振动 RMS 用于判断机械冲击，油液 NAS 用于判断润滑污染，RUL 和复检周期用于形成维护计划。这样的页面结构有助于运维人员快速理解设备状态。

5.3.3 HMI 人机交互页面

齿轮箱 HMI 权限登录页面如图 5.5 所示。该页面面向现场监控场景，在进入控制台前要求用户确认账号、密码和权限级别。

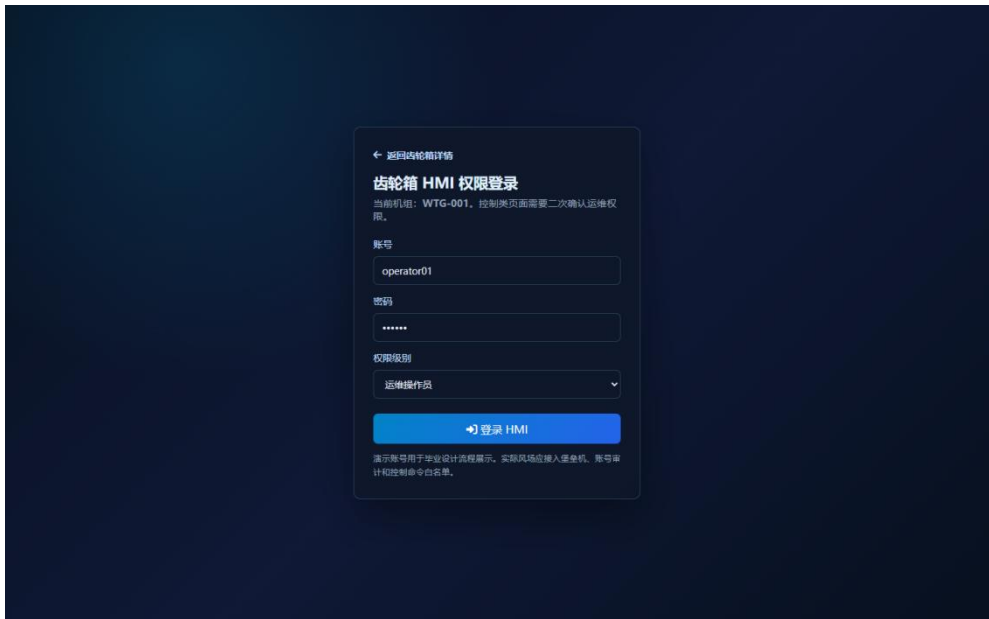


图 5.5 齿轮箱 HMI 权限登录页面

用户完成 HMI 权限登录后进入运行总览页面，如图 5.6 所示。该页面左侧为 HMI 功能菜单，顶部显示当前机组和运行状态，中部展示油温、振动、健康评分、RUL 等关键指标，下方提供运行总览数据。



图 5.6 HMI 登录后运行总览页面

HMI 登录后页面将单机关键指标和控制入口集中到一个界面中，便于运维人员在确认权限后快速查看齿轮箱状态。阈值设置和故障代码页面进一步提供配置与排查入口，其中阈值设置与后端 settings_route.py 参数接口关联，故障代码页面与 data_route.py 的故障码和故障状态数据关联。

HMI 阈值设置页面如图 5.7 所示。该页面用于配置油温、轴承温度和振动阈值，使现场人员能够在登录后根据运行策略调整告警边界，并与后台参数接口保持一致。



图 5.7 HMI 阈值设置页面

HMI 页面主要用于展示齿轮箱关键运行信息和人机交互状态。与普通 Web 仪表盘相比，HMI 页面更强调现场监控感、权限确认和设备对象感，适合在答辩演示时说明系统具备工业监控界面的扩展能力。页面可根据机组编号加载不同设备状态，并与后端接口保持一致的数据来源；阈值设置页面还能根据操作员、工程师或管理员权限决定是否允许修改关键阈值。

通过风场总览、单机详情和 HMI 页面的组合，系统形成了三层展示结构：风场总览用于宏观监控，单机详情用于设备分析，HMI 页面用于现场交互展示。这种结构较符合风电场运维从场站到机组再到部件的业务层级。

代码清单 5.3 HMI 权限登录与菜单切换逻辑

核心代码如下：

```
document.getElementById('hmi-login-form').addEventListener('submit', async (event) => {
    event.preventDefault();

    hmiRole = document.getElementById('hmi-role')?.value || 'operator';
    document.getElementById('login-shell').style.display = 'none';
    document.getElementById('hmi-app').style.display = 'grid';
    await loadDetail();
});
document.getElementById('hmi-menu').addEventListener('click', (event) => {
    const button = event.target.closest('button[data-panel]');
    if (!button) return;
    document.querySelectorAll('#hmi-menu button').forEach(item =>
```

```

        item.classList.toggle('active', item === button));
    renderDetail(button.dataset.panel);
  });

```

代码清单 5.3 位于 frontend/hmi.html。HMI 页面先显示权限登录表单，提交后隐藏 login-shell 并显示 hmi-app，然后加载单机详情、故障代码和阈值参数。左侧菜单通过 data-panel 区分运行总览、状态寿命、阈值设置、故障代码等面板，点击菜单后调用 renderDetail 刷新当前区域，使登录后的各页面能够在同一个 HMI 控制台中切换。

代码清单 5.4 HMI 阈值修改与系统参数同步逻辑

核心代码如下：

```

async function saveHmiThreshold(key, rawValue) {
  const meta = hmiThresholdMeta[key];
  const value = Number(rawValue);
  if (!Number.isFinite(value) || value < meta.min || value > meta.max) {
    alert(`${meta.label}需在 ${meta.min} - ${meta.max} 范围内`);
    renderDetail('params');
    return;
  }
  const next = { ...hmiThresholds, [key]: value };
  if (next.temp_warning_threshold >= next.temp_threshold ||
    next.vibration_threshold >= next.vibration_critical_threshold ||
    next.oil_warning_threshold >= next.oil_quality_threshold) return;
  hmiThresholds = next;
  await fetch('/api/settings/configs', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ [key]: value })
  });
  renderDetail('params');
}

```

代码清单 5.4 对应 HMI 阈值设置页面。系统先根据 hmiThresholdMeta 校验输入范围，再检查关注阈值与临界阈值的大小关系；通过校验后，将当前阈值提交到/api/settings/configs。这样 HMI 页面中的阈值调整不是单纯的前端显示，而是会同步影响系统参数、告警判断和后续健康报告。

5.4 故障代码库与数据管理实现

故障代码库位于 routes/data_route.py。系统内置 15 个齿轮箱关联故障项，如齿轮箱油温过高、齿轮箱油温趋势异常、高速轴承温度异常、振动 RMS 超限、齿轮啮合异常、齿面点蚀/剥落、齿轮断齿风险、润滑油污染/劣化、冷却系统故障、轴系不对中、箱体/基础松动和 RUL 低于预警阈值等。每个故障项包含故障码、名称、类别、关联信号、单位、关注阈值、临界阈值、来源和建议措施。

系统还根据历史故障信息统计动态生成现场故障码。对于不在齿轮箱内置映射中的故

故障码，系统生成 SCADA- 前缀的故障项，并根据标签判断故障类别和严重度。故障码状态根据当前值与阈值判断为正常、关注、临界或未知。

故障数据管理通过 FaultRecord 表和风机文件故障记录共同实现。诊断模块产生的记录会写入数据库；若风机数据文件存在，系统也会根据历史故障信息生成记录列表。系统支持 CSV 导出，导出字段包括机组编号、发生时间、故障类型、严重程度、置信度、处理状态、工单动作和建议措施。

健康报告页面如图 5.8 所示。系统根据当前机组状态和历史故障统计生成健康评分、关键指标、诊断结论和维护建议。



图 5.8 健康报告页面

健康报告页面将实时监测和故障诊断结果转化为面向运维管理的文档化信息。报告中包括机组编号、统计范围、生成时间、健康评分、油温、振动 RMS、油液 NAS、功率、数据质量、RUL 和复检周期等关键指标。相比单纯的仪表盘，健康报告更适合用于阶段性检查和汇报。

报告结论部分会根据健康评分和风险因子生成文字说明，指出当前机组的主要风险和维持建议。如果系统发现油温、振动或油液指标异常，报告会提示缩短复检周期或安排现场检查。该功能体现了智能健康管理体系从监测数据到管理决策的转化能力。

5.5 AI 运维问答模块实现

智能运维诊断助手位于 frontend/index.html、frontend/js/main.js、routes/qa_route.py 和

services/rag_service.py。更新后的页面由当前机组上下文、诊断对话、输出要求和智能程度提示四部分组成，能够把油温、振动 RMS、油液 NAS、RUL、健康评分和功率等实时工况随问题一并提交给后端问答接口。

代码清单 5.6 智能运维诊断助手流式问答逻辑

核心代码如下：

```
async function sendQuestion(customQuestion = null) {
  const question = customQuestion || qaInput.value.trim();
  addMessage(question, true);
  const deepThinking = qaDeepThinking ? qaDeepThinking.checked : true;
  const streaming = createStreamingAssistantMessage();
  let streamedAnswer = '';
  let streamMeta = {};
  const response = await fetch('/api/qa/ask_stream', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      question: question,
      current_status: qaCurrentStatusPayload(),
      deep_thinking: deepThinking,
      answer_mode: deepThinking ? 'model_first' : 'local_only'
    })
  });
  await readSseResponse(response, {
    delta(payload) {
      streamedAnswer += payload.text || '';
      appendStreamingText(streaming.streamBox, payload.text || '');
    },
    done(payload) { streamMeta = { ...streamMeta, ...payload }; }
  });
  finalizeStreamingMessage(streaming, streamedAnswer, {
    sources: streamMeta.sources || [],
    confidence: streamMeta.confidence,
    riskLevel: streamMeta.risk_level,
    engine: streamMeta.engine
  });
}
```

当用户提出问题时，前端先将问题显示为用户消息，再创建流式助手消息，并把当前机组状态、是否启用“大模型主答”和回答模式一并提交到 /api/qa/ask_stream。后端返回 SSE 数据流，前端按 delta 事件逐步追加文本，按 done 事件补充置信度、风险等级、证据来源和后续追问，使回答过程更接近真实智能助手交互。

智能运维诊断助手页面如图 5.9 所示。用户可以围绕当前机组状态、停机判断、振动诊断、油温排查、工单生成和算法解释提出问题，系统结合实时工况、本地知识库和大模型接口生成结构化处置建议。



图 5.9 智能运维诊断助手页面

智能运维诊断助手面向运维人员的日常咨询和答辩演示场景。左侧上下文面板用于确认当前机组和关键指标，中间诊断对话区用于显示用户问题、助手流式回答和证据标签，右侧输出要求区明确回答必须包含结论、依据、排查步骤和工单判断。该设计使问答结果不只是文本回复，而是与齿轮箱健康管理业务流程相连接。

在实现上，系统默认使用大模型主答、本地知识库作为证据补充；当大模型接口不可用时，后端会回退到本地专家规则，保证演示稳定性。回答渲染函数会把“结论、风险等级、关键依据、建议步骤、后续追问”等标题转换为卡片化结构，并提供查看实时趋势、故障记录和健康报告的跳转按钮，增强运维闭环能力。

5.6 健康报告与工单闭环实现

健康报告接口位于 `routes/report_route.py`。系统根据当前机组状态、历史趋势和最新故障记录生成报告。报告内容包括报告编号、机组编号、统计周期、生成时间、健康评分、摘要、关键指标、诊断结果、维护计划和工单动作。当接入风机数据文件时，报告会展示数据范围、平均油温、振动峰值、油液 NAS、有功功率、数据质量、RUL、复检周期、报警级别和故障记录数量。

工单模拟接口位于 `routes/ops_route.py`。当诊断结果为警告或严重时，系统可生成 P2 或 P1 工单，包含工单编号、机组编号、故障类型、优先级、阶段、责任人、创建时间和闭环流程。闭环流程包括诊断触发、生成工单、现场复核、检修处理和复测验收。

系统参数设置页面如图 5.10 所示。该页面用于配置油温、振动、油液阈值和系统访问参数，阈值变化会影响实时告警和健康报告。

参数设置页面与健康报告、故障诊断和工单闭环存在直接关联。油温阈值、振动阈值和油液阈值改变后，系统在实时监测页面中的状态颜色、诊断模块中的风险因子、报告模块中的维护建议都会随之变化。因此，参数设置并不是简单的后台配置，而是健康管理体系统中连接管理策略和诊断结果的关键环节。

在运维闭环中，系统首先根据阈值和风险因子判断异常，再生成诊断结果和维护建议；当风险等级较高时，系统进一步模拟生成工单信息，并把处理状态写入故障记录。这样可以把“发现异常、解释原因、安排处理、复测确认”的过程串联起来，使论文中的系统实现不只停留在页面展示，而是具备完整业务流程。



图 5.10 系统参数设置页面

代码清单 5.5 系统参数阈值校验与保存接口

核心代码如下：

```
@settings_bp.route("/configs", methods=["POST"])
def update_configs():
    data = request.json or {}
    existing = {c.config_key: c.config_value for c in SystemConfig.query.all()}
    candidate = {**defaults, **existing,
                  **{k: v for k, v in data.items() if k in ALLOWED_CONFIGS}}
    for warning_key, critical_key in threshold_pairs:
        warning_value = float(candidate.get(warning_key, 0))
        critical_value = float(candidate.get(critical_key, 0))
```

```

    if warning_value >= critical_value:
        return jsonify({"status": "error",
                        "message": "关注阈值必须小于告警阈值。"}), 400
    for key, value in data.items():
        if key not in ALLOWED_CONFIGS:
            continue

        numeric_value = float(value)

        min_value, max_value = CONFIG_RANGES[key]
        if not min_value <= numeric_value <= max_value:
            return jsonify({'status': 'error'}), 400

        config = SystemConfig.query.filter_by(config_key=key).first()
        config.config_value = str(value)

    db.session.commit()
    return jsonify({"status": "success", "configs": updated})

```

代码清单 5.5 位于 backend/routes/settings_route.py。后端接口只允许 ALLOWED_CONFIGS 中的参数被修改，并通过 CONFIG_RANGES 限制每个阈值的取值范围；同时用 threshold_pairs 保证关注阈值小于告警阈值。该接口是系统参数页面和 HMI 阈值设置页面的共同后端入口，保证不同前端入口写入的是同一套系统配置。

系统设置页面用于体现健康管理体的可配置性。不同风场、不同季节和不同运行策略下，油温、振动和油液阈值可能需要调整。如果阈值完全写死在代码中，系统很难适应现场环境。因此系统提供参数配置页面，使管理员能够根据运维策略调整预警阈值和严重阈值。

参数配置不仅影响页面显示，也会影响诊断结果、告警等级和报告建议。这样可以保证系统的业务逻辑与管理策略一致。用户管理和角色权限则用于控制不同人员的操作范围，避免普通用户误改关键阈值。

第 6 章 系统测试与结果分析

6.1 测试环境

系统测试环境为 Windows 本地环境，后端使用 Python 虚拟环境运行 Flask 服务，浏览器访问 `http://127.0.0.1:5000`。后端依赖包括 Flask、Flask-CORS、Flask-SQLAlchemy、NumPy、Pandas、scikit-learn、python-dotenv、xlrd 和 OpenAI 兼容库等。前端依赖 ECharts、Three.js 和 Font Awesome。

6.2 功能测试

表 6.1 系统功能测试结果

测试项目	测试内容	结果
登录注册	管理员登录、新用户注册和角色显示	通过
风场总览	多机组状态、统计指标和机组卡片展示	通过
单机详情	切换机组后关键指标同步变化	通过
实时监测	SSE 推送油温、振动、功率和波形数据	通过
故障诊断	不同故障场景输出诊断结果和建议	通过
报告导出	生成健康报告并导出故障记录 CSV	通过
运维问答	基于知识库和当前工况生成排查建议	通过

登录注册测试中，系统能够完成管理员账号登录和新用户注册，并在前端显示用户角色。风场总览测试中，系统能够展示多机组运行状态、风场统计指标和状态图例。单机详情测试中，选择不同机组后，油温、功率、风速、健康评分、RUL、告警和集群对标能够随之变化。

实时监测测试中，系统通过 SSE 持续推送状态数据和振动波形，前端能够刷新油温、振动、功率、RUL、采集链路和图表。故障诊断测试中，选择不同场景后，系统能够生成相应故障类型、风险等级、置信度、健康评分、RUL、风险因子和维护建议。故障数据测试中，系统能够显示诊断记录和历史故障记录，并支持导出 CSV。

故障代码库测试中，系统能够根据当前值和阈值判断故障项状态，并支持按机组展示。

健康报告测试中，系统能够生成报告摘要和关键指标。AI 问答测试中，系统能够回答油温过高、振动异常、轴承故障、齿轮故障、油液污染、检修周期和算法原理等问题。

6.3 诊断场景测试分析

系统提供多种诊断场景。正常运行场景中，油温、振动和油液质量均处于较低风险范围，系统输出正常运行，健康评分较高，维护建议为保持日常巡检。齿轮齿面磨损场景中，振动能量和油液污染风险轻度升高，系统输出齿轮齿面磨损，并建议 72 小时内复测振动趋势和取油样复核。

齿面点蚀/剥落和齿轮断齿场景中，系统通过加入周期性冲击提高峭度、峰值因子和包络峰值，输出严重风险，并建议降载或停机检查。轴承点蚀/剥落场景中，冲击特征明显升高，系统建议结合 120-240Hz 特征频段和包络谱进行复核。油温过高和冷却系统故障场景中，油温风险因子升高，系统建议检查散热器、油冷风扇、三通阀、温控开关、油位和滤芯压差。

油液污染场景中，油液 NAS 指标升高，系统建议取样复测 NAS 等级、水分、黏度和金属颗粒，并检查滤芯、呼吸器和密封。轴系不对中和基础松动场景中，低频振动增强，系统建议复核联轴器、地脚螺栓、弹性支撑和基础连接。

6.4 结果评价

从功能完整性看，系统覆盖了毕业设计要求的登录、系统管理、用户管理、故障数据、数据分析、参数设置、故障分析、AI 智能问答和知识库等内容。从工程流程看，系统能够形成从数据采集、特征提取、诊断判断、健康评分、RUL 估计、建议生成、记录保存到报告和工单的闭环。从展示效果看，系统页面较适合风电运维场景，具有多机组总览、单机下钻、图表分析和 HMI 展示能力。

系统不足也较明显。当前诊断算法主要是规则模型和模拟信号，尚未完成真实 M-IALO-SVR 模型训练和大规模故障样本验证。前端展示中提到的 M-IALO-SVR、VMD-CNN、包络阶次分析和小波散度等内容有一定演示性质，正式论文答辩时需要说明系统当前处于原型阶段，并强调已预留真实模型接入位置。此外，系统尚未进行高并发、长时间运行和生产安全审计测试。

从运行效果看，系统界面能够支持从风场总览到单机详情的下钻分析，故障诊断结果具有一定可解释性。系统不足在于当前数据以模拟数据和统计文件为主，后续仍需接入真实 SCADA、CMS 和油液监测数据，并利用更多真实故障样本对模型进行训练和校验。

测试结果表明，系统能够完成风场总览、单机详情、实时监测、故障诊断、故障数据管理、健康报告和 AI 运维问答等主要功能。对于油温升高、振动增强、油液污染和轴承冲击等典型场景，系统能够给出风险等级、故障类型、健康评分、RUL 估计和维护建议。

第7章 总结与展望

7.1 研究总结

系统实现时采用本地可运行的 Web 架构。后端使用 Flask 组织认证、风场、数据、报告、问答和运维接口，数据库使用 SQLite 保存用户、故障记录、参数配置和闭环处理信息；前端由 HTML、CSS 和 JavaScript 组成，利用 ECharts 展示趋势和统计图，用 Three.js 展示齿轮箱结构示意，并通过 SSE 接收实时状态。项目代码中既有 DataAcquisitionSystem 生成的油温、振动、油液和功率测点，也有 WindDataRepository 对风机数据文件的整理结果，因此页面能够同时显示模拟实时数据和历史故障统计信息。

在算法流程方面，系统实现了振动特征提取、风险因子计算、故障分类、健康评分和 RUL 估计。系统能够识别齿轮齿面磨损、齿面点蚀/剥落、齿轮断齿、轴承磨损、轴承点蚀/剥落、高速轴承过温、齿轮箱油温过高、润滑油污染/油液劣化、轴系不对中、齿轮啮合异常、箱体或基础松动和冷却系统故障等常见异常，并给出诊断依据和维护建议。

在工程实现方面，系统将后端接口、数据库模型、数据仓库、诊断服务、知识库和前端界面整合在一起，形成较完整的毕业设计原型。测试结果表明，系统能够完成主要业务流程，具备一定的演示价值和工程参考意义。

7.2 存在不足

本文系统仍存在以下不足：第一，诊断模型以规则和模拟数据为主，缺少真实故障样本训练和验证；第二，M-IALO-SVR、VMD-CNN 等算法尚未形成完整训练代码和评价结果，更多体现为系统预留和方法说明；第三，健康评分和 RUL 估计采用经验规则，尚未经过长期运行数据校准；第四，系统权限审计、数据安全、工单闭环和生产部署能力仍有待加强。

7.3 后续展望

后续研究可从以下方向继续完善。首先，接入真实 SCADA、CMS、油液检测和检修工单系统，建立标准化数据字典和样本库。其次，基于真实数据训练 M-IALO-SVR 油温残差预测模型，并与 SVR、随机森林、XGBoost、LSTM 等方法进行对比。再次，引入更多模型评价指标，如 MAE、RMSE、R2、准确率、召回率、F1 值和混淆矩阵，全面评价模型效果。最后，完善告警确认、检修派单、复测验收、权限审计和报表归档功能，使系统逐步从毕业设计原型向真实风电场运维辅助平台发展。

参考文献

- [1] 杨俊峰.小样本下基于深度神经网络的行星齿轮箱故障诊断[D].成都:电子科技大学,2024.DOI:10.27005/d.cnki.gdzku.2024.005469.
- [2] 周昶清.基于统计模型与稀疏表示的齿轮箱故障检测与诊断方法研究[D].杭州:浙江大学,2024.DOI:10.27461/d.cnki.gzjdx.2024.000307.
- [3] 武甲.基于深度学习与降维技术的风电机组齿轮箱状态监测可视化和健康评估[D].呼和浩特:内蒙古工业大学,2024.DOI:10.27225/d.cnki.gnmgu.2024.000206.
- [4] 许晴.基于时频分析和机器学习的齿轮箱故障诊断[D].北京:华北电力大学(北京),2024.
- [5] 吴岚.基于先进信号分析方法的风电机组齿轮箱故障诊断[D].北京:华北电力大学(北京),2024.
- [6] 马健.基于振动信号的风电机组齿轮箱特征提取方法和故障诊断研究[D].上海:上海电机学院,2024.
- [7] 吴迪.基于自动特征学习的风机齿轮箱故障诊断研究[D].沈阳:沈阳工业大学,2024.DOI:10.27322/d.cnki.gsgyu.2024.000493.
- [8] 孔德强.风电齿轮箱温度预警及关键部件故障诊断方法[D].沈阳:沈阳工程学院,2024.DOI:10.27845/d.cnki.gsygc.2024.000003.
- [9] 李常见.风电机组齿轮箱故障识别及安全状态评估研究[D].阜新:辽宁工程技术大学,2024.
- [10] 郭建超.风电机组齿轮箱轴承微弱故障诊断方法研究及系统开发[D].上海:上海电机学院,2024.
- [11] 王广.风力发电机组齿轮箱故障检测和寿命预测方法的研究[D].兰州:兰州理工大学,2024.
- [12] 范亚飞.基于变分模态分解和随机配置网络的齿轮箱故障诊断研究[D].石家庄:石家庄铁道大学,2024.DOI:10.27334/d.cnki.gstdy.2024.000257.
- [13] 杨青松.基于改进 ShuffleNet 的齿轮箱故障诊断研究[D].石家庄:石家庄铁道大学,2024.DOI:10.27334/d.cnki.gstdy.2024.000304.
- [14] 王莘然.基于迁移学习的风电齿轮箱故障诊断方法研究[D].大连:大连海事大学,2024.
- [15] 王丹.基于深度迁移学习的齿轮箱故障诊断方法研究[D].济南:齐鲁工业大学,2024.DOI:10.27278/d.cnki.gsdqc.2024.000503.

致谢

本毕业设计从选题、需求分析、系统设计、代码实现到论文撰写，得到了指导教师的悉心指导和同学的帮助。在完成课题过程中，本人进一步学习了风电机组齿轮箱故障诊断、Python Web 开发、数据库设计、前端可视化、数据分析和智能运维等知识。感谢指导教师研究方向、功能设计和论文结构方面提出的宝贵意见，感谢同学在系统测试和问题反馈中的支持。由于本人水平有限，论文和系统仍存在不足之处，恳请各位老师批评指正。